Atelier pratique n°3 : les bases de Git (réponses)

Stagiaire : Ali EL Habib · OFPPT Tanger · Développement Digital
Dépôt : `atelier-git` (3 commits demandés + 1 commit pour ce fichier de réponses)

Partie 1 : configuration et création du projet

Commandes exécutées :

```
git --version
git config --global user.name "Ali EL Habib"
git config --global user.email "mon.email@exemple.com"
mkdir atelier-git
cd atelier-git
git init
git status
```

À quoi sert la commande `git init` ? Elle transforme le dossier courant en dépôt Git : elle crée le dossier caché `.git`, qui contiendra tout l'historique (commits, branches, configuration locale). Avant `git init`, Git ignore complètement le dossier.

Quel est le rôle des commandes `git config` ? Elles enregistrent l'identité de l'auteur (nom et email). Git écrit ces informations dans chaque commit pour savoir *qui* a fait *quoi*. Avec `--global`, la configuration vaut pour tous les dépôts de l'ordinateur.

Que signifie l'expression *working directory* ? C'est le répertoire de travail : les fichiers tels qu'ils sont actuellement sur le disque, ceux qu'on ouvre et modifie dans l'éditeur. Les changements faits ici ne sont pas encore enregistrés dans Git tant qu'on ne les a pas ajoutés (`git add`) puis validés (`git commit`).

Partie 2 : suivi des fichiers et premier commit

```
# création de index.html avec le titre « Mon premier projet Git »
git status                                  # index.html apparaît en "Untracked"
git add index.html
git status                                  # index.html apparaît en "Changes to be
committed"
git commit -m "Initialisation du projet"
git log
```

Quelle est la différence entre `git add` et `git commit` ? `git add` place les modifications dans la zone de staging : on choisit ce qui fera partie de la prochaine version. `git commit` enregistre ensuite cette sélection comme une version définitive dans l'historique, avec un identifiant, un auteur, une date et un message.

Quel est le rôle de la zone de staging ? C'est une zone intermédiaire entre le répertoire de travail et l'historique. Elle permet de préparer précisément le contenu d'un commit, par exemple n'enregistrer que deux fichiers sur cinq modifiés, pour obtenir des commits cohérents.

À quoi sert l'option `-m` ? Elle donne le message du commit directement dans la commande. Sans `-m`, Git ouvre un éditeur de texte pour saisir le message.

Partie 3 : modifier un fichier

```
# ajout d'un paragraphe de présentation dans index.html
git status          # index.html est "modified"
git diff            # montre les lignes ajoutées (+) dans le répertoire de travail
git add index.html
git diff --staged   # montre ce qui est dans la zone de staging
git commit -m "Ajout de la présentation"
git log --oneline
```

Que montre `git diff` ? Les différences ligne par ligne entre le répertoire de travail et la zone de staging, c'est-à-dire les modifications **pas encore ajoutées** avec `git add`. Les lignes ajoutées commencent par `+` et les lignes supprimées par `-`.

Quelle est la différence entre `git diff` et `git diff --staged` ? - `git diff` : working directory comparé à la zone de staging (changements non ajoutés). - `git diff --staged` : zone de staging comparée au dernier commit (ce qui partira dans le prochain commit).

Après `git add index.html`, `git diff` n'affiche plus rien, alors que `git diff --staged` affiche le paragraphe ajouté.

Pourquoi créer plusieurs commits au lieu d'un seul ? Chaque commit représente une étape logique. On obtient un historique lisible, on peut retrouver quand et pourquoi une modification a été faite, et on peut annuler une seule étape sans perdre tout le reste.

Partie 4 : gérer plusieurs fichiers

```
# création de style.css et README.md
git status                          # les deux fichiers sont "Untracked"
git add style.css README.md         # deux fichiers en une seule commande
git status
git commit -m "Ajout du style et de la documentation"
git log --oneline
```

Quelle est la différence entre un fichier *tracked* et *untracked* ? - **Tracked (suivi)** : le fichier fait déjà partie d'au moins un commit (ou a été ajouté au staging). Git surveille ses modifications. - **Untracked (non suivi)** : le fichier existe dans le dossier mais Git ne l'a jamais enregistré. Il ne sera pas versionné tant qu'on ne fait pas `git add`.

Peut-on ajouter plusieurs fichiers avec une seule commande `git add` ? Oui. On peut les lister (`git add style.css README.md`), utiliser un motif (`git add *.css`) ou tout ajouter (`git add .`).

À quoi sert le fichier `README.md` ? C'est la documentation d'accueil du projet, écrite en Markdown. Elle explique ce qu'est le projet, comment l'utiliser ou l'installer. GitHub l'affiche automatiquement sur la page du dépôt.

Partie 5 : bilan et vérification

Commande	Rôle
`git init`	Crée un nouveau dépôt Git (dossier `.git`) dans le dossier courant.
`git status`	Affiche l'état du dépôt : branche, fichiers modifiés, en staging ou non suivis.
`git add`	Ajoute des modifications à la zone de staging pour le prochain commit.
`git commit`	Enregistre le contenu du staging comme une nouvelle version dans l'historique.
`git log`	Affiche l'historique des commits (auteur, date, message, identifiant).
`git diff`	Montre les différences ligne par ligne entre deux états des fichiers.

Livrable

- Dossier `atelier-git` contenant `index.html` , `style.css` et `README.md` : fait.
- Dépôt Git local initialisé : fait (puis publié sur GitHub).
- Au moins trois commits avec des messages explicites :
    1. `Initialisation du projet`

2. `Ajout de la présentation`
3. `Ajout du style et de la documentation`

- Historique obtenu avec `git log --oneline` : voir le fichier historique.txt.